

[Bài đọc] Custom Validator

1. Giới thiệu

Ngoài các annotation như `@NotNull`, `@Valid`, Spring còn cung cấp interface `Validator` để bạn có thể tự tạo class kiểm tra tính hợp lệ của dữ liệu **một cách linh hoạt** hơn, không cần annotation.

Interface này thường được dùng khi:

- Bạn không dùng Bean Validation (JSR-380)
- Cần kiểm tra **đa trường** hoặc logic phức tạp
- Muốn kiểm soát lỗi validation trong controller hoặc service

2. Giao diện

`org.springframework.validation.Validator`

```
public interface Validator {  
    boolean supports(Class<?> clazz);  
    void validate(Object target, Errors errors);  
}
```

▪ `supports(Class<?> clazz)`

- Trả về `true` nếu validator hỗ trợ validate cho class đó.
- Spring dùng method này để biết validator nào áp dụng cho object nào.

▪ `validate(Object target, Errors errors)`

- Chứa logic kiểm tra dữ liệu.
- Nếu có lỗi → gọi `errors.rejectValue(...)` hoặc `errors.reject(...)`

3. Ví dụ: Tạo custom validator cho UserDTO

◆ Bước 1: Tạo DTO

```
private String password;  
  
// getters, setters  
}
```

◆ Bước 2: Tạo class validator

```
@Component  
public class UserValidator implements Validator {  
  
    @Override  
    public boolean supports(Class<?> clazz) {  
        return UserDTO.class.equals(clazz);  
    }  
  
    @Override  
    public void validate(Object target, Errors errors) {  
        UserDTO user = (UserDTO) target;  
  
        // Kiểm tra trống  
        ValidationUtils.rejectIfEmptyOrWhitespace(errors, "username", "username.empty", "Username is required");  
        ValidationUtils.rejectIfEmptyOrWhitespace(errors, "password", "password.empty", "Password is required");  
  
        // Kiểm tra Logic riêng  
        if (user.getPassword() != null && user.getPassword().length() < 6) {  
            errors.rejectValue("password", "password.short", "Password must be at least 6 characters");  
        }  
  
        if (user.getUsername() != null && user.getUsername().contains(" ")) {  
            errors.rejectValue("username", "username.space", "Username cannot contain spaces");  
        }  
    }  
}
```

◆ Bước 3: Sử dụng trong Controller

```
@Controller
public class UserController {

    private final UserValidator userValidator;

    public UserController(UserValidator userValidator) {
        this.userValidator = userValidator;
    }

    @PostMapping("/register")
    public String register(@ModelAttribute("user") UserDTO user, BindingResult result) {
        userValidator.validate(user, result);

        if (result.hasErrors()) {
            return "registerForm"; // quay lại form hiển thị lỗi
        }

        // Xử lý đăng ký
        return "success";
    }
}
```

4. Ưu điểm của Spring Validator

Không cần annotation

Có thể tái sử dụng cho nhiều controller/form

Kiểm tra nhiều trường, nhiều logic tùy biến

Có thể dùng `MessageSource` để đa ngôn ngữ hóa error message